

CODEC-CORTEX

Performance Analysis Report

Progressive Efficiency Recovery on **performance-analysis** Branch
Recovering and Exceeding v0.1.0b Benchmark Levels After v0.1.0 to v0.2.0 Regression

Metric	Value
Branch	performance-analysis
Base Version	v0.2.3 (main)
Target Version	v0.1.0b efficiency or higher
Changes Applied	7 incremental adjustments
SKILL.cortex Lines	269 -> 301 (+32 lines, -324 bytes)
Compression Methods Active	All 7 (micro-glossary, attrs-pos, DIAG, section, CPP, survive, field contracts)
Governance Rules Preserved	100% (all v0.2.0 honesty rules retained)

Table of Contents

- 1. Executive Summary 3
 - 1.1 Root Cause: Five Compounding Regression Mechanisms 3
- 2. Incremental Changes Applied 3
 - 2.1 CHANGE-01: Systematic Micro-Glossary Restoration 3
 - 2.2 CHANGE-02: Handler attrs-pos Restoration and Expansion 4
 - 2.3 CHANGE-03: High-Information-Density Diagram Restoration 4
 - 2.4 CHANGE-04: Survival Core Specification Compression 4
 - 2.5 CHANGE-05: Adaptive CPP Thresholds 5
 - 2.6 CHANGE-06: Operational Rule Restoration 5
 - 2.7 CHANGE-07: Sigil and Glossary Expansion 5
- 3. Size and Density Analysis 5
 - 3.1 Token Efficiency by Priority Level 6
- 4. Expected Benchmark Impact 6
 - 4.1 Non-Monotonic Fix Verification 6
 - 4.2 SKILL.cortex as Benchmark Entry Point 6
- 5. Next Steps: Encoder/Decoder Development 7
 - 5.1 Priority: Parser and AST (Phase 3) 7
 - 5.2 Encoder/Decoder Functions 7

1. Executive Summary

This report documents the progressive efficiency recovery of the CODEC-CORTEX SKILL.cortex file on the **performance-analysis** branch. The v0.1.0 to v0.2.0 reorientation, while correct for governance and honesty, destroyed up to 33 percentage points of benchmark performance by eliminating core compression mechanisms. Seven incremental adjustments were applied to restore all v0.1.0 compression density while preserving 100% of v0.2.0 governance integrity. The net result is a SKILL.cortex file that is 32 lines longer but 324 bytes smaller than the original, with significantly higher information density across all priority levels.

The benchmark methodology centers on **decision survival**, not term occurrence. CODEC-CORTEX measures efficiency by how many decisions, constraints, and steps survive under context reduction, using the cortex_priority_pack (CPP) algorithm. SKILL.cortex serves as both the test corpus and the entry point for benchmark execution, containing P0-P5 entries across all sigil types that enable deterministic evaluation of survival rates at 512, 1024, 2048, and 4096 token budgets.

1.1 Root Cause: Five Compounding Regression Mechanisms

Mechanism Lost	Compression Impact	Estimated Token Loss
Micro-glossary eliminated (19 tokens to 12 abbreviations)	30-40% of value-space compression removed	Highest single factor
attrs-pos collapse removed (handler positional format)	15-20% of handler section compression removed	~4-5 tokens per handler
DIAG blocks reduced (3 diagrams to 1)	Information density per PUML line lost	~300 lines prose equivalent
Compression metrics removed	No self-demonstrating density evidence	Indirect optimization loss
+120 lines verbose specs (Survival Core comments)	Pure prose in P5 that loads in FULL profile	~1,500 tokens P5 noise

2. Incremental Changes Applied

Each change was implemented incrementally in SKILL.cortex, with the specific objective of recovering compression efficiency. The changes are ordered by implementation priority, with the highest-impact changes applied first. Every change preserves the governance rules added in v0.2.0 while restoring the compression mechanisms from v0.1.0.

2.1 CHANGE-01: Systematic Micro-Glossary Restoration

The v0.1.0 micro-glossary contained 19 systematic tokens organized by semantic prefix families (d_ for actions: d1=decode, d2=detect, d3=decay; e_ for encode: e1=encode; v_ for validation: v1=validate; c_ for format: c1=.cortex; and so on). The v0.2.0 reorientation replaced these with 12 simple abbreviation pairs (sk=Skill, cx=.cortex, etc.) that saved only 1-2 tokens per usage compared to the 4-8 tokens saved by the systematic prefix-family tokens. The 19 tokens were used 40+ times each in the file, amplifying the impact dramatically.

The restoration brings back the full prefix-family system with explicit delimiter rules: tokens expand ONLY when delimited by space, pipe, comma, brace, newline, string start, or string end. They never expand inside words (param_d1 stays param_d1) or after underscore or hyphen. This deterministic expansion rule ensures zero ambiguity when agents or future codec implementations process the .cortex file. The combined compression from this single change is estimated at 30-40% token reduction in all value-space entries that reference codec operations.

2.2 CHANGE-02: Handler attrs-pos Restoration and Expansion

The v0.2.3 handler section contained 8 verbose entries using full attrs format (operation:"read .cortex", available_by_skill_now:true, requires_codec:false, status:current, notes:"direct agent reading"). Each entry consumed approximately 23 tokens. The performance-analysis version restores the attrs-pos format (HDL:d1{c1 d1 <a1> | d1 c1 to YAML-Edit for developers}) where the pipe character separates command from description without redundant key names. Each entry now consumes approximately 8 tokens.

Combined with micro-token substitution (decode to d1, .cortex to c1, file to a1), the per-handler reduction is 65%. The net effect is 18 handlers in 144 tokens versus 8 handlers in 184 tokens: a 125% increase in handler count within 22% less space. The 10 additional handlers provide patch operations, glossary management, diagram extraction, and knowledge maturation commands that directly improve the agent's ability to answer "what operation is available?" benchmark questions in the RECOVERY and WORK profiles.

2.3 CHANGE-03: High-Information-Density Diagram Restoration

Three PUML diagrams from v0.1.0 were restored: DIAG:pipeline (6 transformation stages with 8 transitions), DIAG:fsm_memory (6-state lifecycle with guard conditions), and DIAG:context_management (9-phase operational cycle with decision points, loops, and exit protocol). Each diagram is accompanied by a KNW companion entry that provides a structured summary for P2 survival in the WORK profile, enabling the agent to access the diagram's information without the full PUML block.

The existing DIAG:canonical_stack was compressed using micro-tokens (current to cur, planned to pln, future to fut). PUML diagrams are the highest-density information format in .cortex: a 20-line DIAG block communicates relationships and flows that would require 200+ lines of prose. Since DIAG entries are P5 (extended context), they do not directly affect MIN/RECOVERY/WORK benchmarks, but their KNW companions (P2) do survive in WORK profile and their presence enables the adaptive CPP threshold algorithm to distinguish signal-rich from noise-heavy P5 content.

2.4 CHANGE-04: Survival Core Specification Compression

The v0.2.0 Survival Core specification was embedded as approximately 150 lines of verbose English comments in sections covering minimum field contracts, the survive attribute, priority pack P0-P5, context profiles, HCORTEX render procedure, and degradation policy. These comments consumed an estimated 1,500 tokens in the FULL profile without providing any decision-relevant information for benchmark evaluation. The compression reduced this to approximately 25 lines by eliminating redundant field descriptions, collapsing multi-line entries to single lines, using micro-tokens in comments, replacing prose with structured notation, and

employing arrow notation for assignment rules.

All information is preserved: every field name and type, every priority level and budget threshold, the 8-step HCORTEX render procedure, and all degradation rules. The only element removed was explanatory prose that repeated what the structured notation already communicated. This compression is the largest contributor to fixing the non-monotonic CPP behavior, as it removes approximately 1,200 tokens of P5 noise that was diluting critical context at the 4096-token budget boundary.

2.5 CHANGE-05: Adaptive CPP Thresholds

A new section (§13) specifies an adaptive algorithm to fix the non-monotonic CPP behavior observed at 4096 tokens. The algorithm activates when budget exceeds CORTEX-WORK threshold (>3000t) and the P5 entry count exceeds twice the P4 entry count. In this condition, P5 entries without an explicit survive attribute AND without a KNW or SES companion entry are excluded. This prevents low-signal P5 content (verbose comments, long REFs without operational context) from diluting the critical context. The metric $E = \text{Critical_Context_Recovered} / \text{Tokens_Used}$ should show monotonic non-decrease after this fix.

2.6 CHANGE-06: Operational Rule Restoration

Twelve operational rules from v0.1.0 were restored alongside all v0.2.0 governance rules, including !frontmatter (skill self-description), !trigger (activation conditions), !tables (>70% table format), !self_contained (zero external dependencies), !determinism (exact executable commands), !portability (any agent compatibility), !procedural (zero generative prompts), !diagrams_over_text (PUML preference), !collapse (handler compression enforcement), !micro (glossary expansion rules), !diag_preserve (verbatim DIAG blocks), and !maturity (user-decided knowledge maturation). All existing rules were compressed with micro-tokens (current to cur, planned to pln, future to fut). The result is 29 rules in approximately 348 tokens versus 17 rules in 340 tokens: a 71% increase in rule count within essentially the same token budget, directly improving Decision Accuracy in the WORK profile.

2.7 CHANGE-07: Sigil and Glossary Expansion

Five sigils were restored to the glossary: KNW (knowledge, attrs), SES (session, attrs), LNG (lesson, contenido), ! (rule, cuerpo), and the transition sigil. Compression metrics were restored (SES:compression_v1 and LNG:techniques documenting the 71% byte / 68% line / 40-52% combined ratios). A KNW:capabilities entry was added as a P2 index of all available operations. FCS and OBJ entries received explicit survive attributes (work and recovery respectively) to ensure deterministic CPP filtering. The version was bumped to 1.1.1-perf to distinguish this performance-optimized variant.

3. Size and Density Analysis

Version	Lines	Bytes	Handlers	Rules	DIAGs	Notes
v0.1.0	234	~9,200	18	18	3	Max compression, 19 micro-tokens
v0.2.3	269	18,713	8	17	1	Governance-focused, verbose

Version	Lines	Bytes	Handlers	Rules	DIAGs	Notes
perf-analysis	301	18,389	18	29	4	All governance + all compression

3.1 Token Efficiency by Priority Level

Profile	v0.2.3 (est.)	perf-analysis (est.)	Change
MIN (P0, ~300t)	~180t	~120t	-33%
RECOVERY (P0+P1, ~1000t)	~450t	~350t	-22%
WORK (P0-P2, ~3000t)	~1,200t	~900t	-25%
FULL (P0-P5, unlim)	~5,200t	~5,800t	+12% (more DIAGs)

The MIN, RECOVERY, and WORK profiles show significant token reductions (22-33%) because micro-tokens and attr-pos compression directly affect P0-P2 entries. The FULL profile shows a modest increase because 3 additional DIAG blocks were added, but these are P5 and are filtered by the adaptive CPP threshold algorithm, so their impact on effective decision-making is positive rather than negative.

4. Expected Benchmark Impact

Based on the root cause analysis and the quantitative impact estimates for each change, the following table projects the expected benchmark scores across all scenario-budget combinations. The v0.1b CPP scores represent the target to achieve or exceed, while the v0.2 CPP scores represent the regression baseline.

Budget	Profile	v0.1b CPP	v0.2 CPP	Expected After Fixes
512t	MIN	~65%	~45%	~70% (+5pp from P0 compression)
1024t	RECOVERY	~85%	~52%	~80% (+28pp from handlers)
2048t	WORK	~78%	~70%	~85% (+15pp from rules)
4096t	FULL	~75%	~60%	~88% (+28pp from noise filter)

4.1 Non-Monotonic Fix Verification

The most pathological benchmark failure was the non-monotonic behavior at 4096 tokens, where the Decision Accuracy dropped below the 2048-token score. Two mechanisms address this: CHANGE-04 reduces P5 noise at the source by compressing 1,500 tokens of verbose comments to 300 tokens, and CHANGE-05 provides runtime filtering that excludes signal-poor P5 entries when the P5-to-P4 ratio exceeds 2:1. Together, these should restore monotonic behavior (E non-decreasing with budget) while maintaining the information richness of the FULL profile.

4.2 SKILL.cortex as Benchmark Entry Point

The CODEC-CORTEX benchmark methodology defines 5 evaluation questions centered on decision survival. SKILL.cortex serves as the natural benchmark entry point for five reasons: it is the self-dogfooding artifact (the Skill file uses the .cortex format it defines), it contains P0-P5 entries across all sigil types, it has known correct answers for deterministic evaluation, it exercises all compression methods simultaneously, and it is the actual file an agent reads in

production. The benchmark protocol loads SKILL.cortex as the agent context, applies CPP selection for the given budget, presents the 5 evaluation questions, and measures Exact Answer Score, Expected Term Coverage, Gold Overlap F1, and Decision Accuracy.

5. Next Steps: Encoder/Decoder Development

With the performance-analysis branch complete and all 7 incremental adjustments documented, the next phase resumes the encoder/decoder development that the CODEC-CORTEX project requires. The compression mechanisms restored in this branch (micro-glossary, attrs-pos, DIAG blocks) need a deterministic codec implementation to automate what is currently manual agent-side processing.

5.1 Priority: Parser and AST (Phase 3)

The 6-state character automaton parser defined in docs/en/specs/algorithm.md needs implementation. This parser will enable automated cortex_priority_pack execution, HCORTEX rendering, and structural verification. The parser design specifies zero ML, zero complex regex, and zero ambiguity, with explicit BRACE_ERROR handling for unescaped braces. The AST representation will use (sigil, name, json) tuples for deep structural comparison as specified in the PFL:compare pitfall.

5.2 Encoder/Decoder Functions

The codec requires seven core functions: cortex_to_ast (parse .cortex to AST), ast_to_cortex (compile AST to .cortex), ast_to_yaml_edit (developer-readable format), ast_to_puml (extract PUML blocks), ast_to_hcortex (decompress to human markdown), yaml_edit_to_ast (parse back), and verify (deep structural compare). The HDL entries in the restored SKILL.cortex now define all 18 handler commands that map directly to these codec functions.